

BASES DE DATOS II

Base de Datos II

Trabajo Práctico — Parcial 2



StreamBase — MVP con MongoDB

Tecnicatura Universitaria en Programación | Div133



mongoDB

N



películas

id
titulo
anio
director
generos
calificacion {...}



usuarios

_id
nombre
email
plan {...}



reproducciones

usuario_id
pelicula_id
duracion_min
completada
fecha {...}

```
db.reproducciones.aggregate([
  { $match: {...} },
  { $lookup: {...} },
  { $unwind: {...} },
  { $group: {...} },
  { $sort: {...} },
  { $project: {...} }
])
```



¿Qué hace este programa?

StreamBase es un MVP (Mínimo Producto Viable) que simula el backend de una plataforma de streaming tipo Netflix. Demuestra el modelado de datos en MongoDB usando decisiones fundamentadas de embedding y referencing, y ejecuta tres Aggregation Pipelines reales sobre la base de datos.



GRUPO 14

Sebastián Lescano
Joaquín Avallone
Lautaro Subeldia
Thiago Martínez
Mateo Corbetto



MODELADO DE DATOS

Embedding y Referencing



AGGREGATION

3 Pipelines reales de consulta



MONGODB

Colecciones y relaciones



MVP FUNCIONAL

Backend de una plataforma de streaming



PYTHON
3.9 o superior



MONGODB
localhost:27017



PYTHON
seed.py
queries.py



GITHUB
LautaroSantiago/
streambase-bd2



AÑO
2026

PARTE 2 — Modelado NoSQL: Diseño y Justificación

1. Criterio de diseño: primero las consultas, después el modelo

Identificamos las operaciones más frecuentes del sistema. En MongoDB, el diseño no parte de la normalización sino de las consultas: se modela en función de cómo se va a leer y escribir la información.

Las consultas principales que el sistema debe responder eficientemente son:

1. Mostrar el catálogo con título, géneros, director y calificación de cada título.
2. Reproducir contenido y registrar el progreso por perfil (retomar donde se dejó).
3. Consultar el historial de visualización de un perfil específico.
4. Mostrar la lista de contenido guardado ("Mi Lista") de un perfil.
5. Filtrar series por temporada y episodio para continuar la reproducción.

Entidades en el sistema y sus relaciones

El sistema está diseñado bajo un enfoque no relacional orientado a documentos (MongoDB), modelando la interacción del servicio de streaming mediante tres colecciones principales:

- **usuarios:** Almacena los datos de identidad de los clientes y los detalles operativos de su suscripción actual.
- **películas:** Funciona como el catálogo central de contenidos disponibles en la plataforma, consolidando toda la información técnica, artística y de clasificación de cada título.
- **reproducciones:** Es una **colección asociativa independiente** que resuelve la relación Muchos a Muchos (\$N:M\$) entre **usuarios** y **películas**. Registra el historial de visualizaciones individuales, guardando métricas específicas de consumo (como duración visualizada, fecha y si se terminó de ver).

¿Dónde se usó Embedding

El *embedding* se aplicó en aquellos escenarios donde existe una relación de dependencia directa

director dentro de películas: Mantiene una relación 1:1 directa con la obra. Dado que la información del director (nombre, nacionalidad) es un atributo descriptivo que se consulta el 100% de las veces junto a los datos de la película (como en la ficha técnica del catálogo), incrustarlo elimina la necesidad de hacer consultas cruzadas.

géneros dentro de películas: Se modeló como un array de strings integrado. Aplica la heurística de "*Limitado/Pequeño $\rightarrow$ incrustar*", ya que una película tiene una cantidad fija y muy baja de géneros (normalmente entre 1 y 5). Nunca crecerá de forma indefinida ni superará el límite físico del documento.

Plan dentro de usuarios: El plan de suscripción (tipo, precio, fecha de inicio) es un atributo del usuario en este modelo y no una entidad independiente con lógica propia. Al estar embebido, la plataforma puede validar de manera inmediata si un usuario tiene su suscripción activa al momento de iniciar sesión mediante una única lectura atómica.

¿Dónde se usó Referencing

El *referencing* (o vinculación por IDs) se utilizó de forma obligatoria en la colección de **reproducciones** para conectar a los **usuarios** con las **películas**:

- **reproducciones hacia usuarios (usuario_id) y películas (pelicula_id):**
 - El historial de reproducciones es una estructura de **expansión constante y sin límites**. Si se decidiera embeber las reproducciones dentro de cada usuario, el documento del usuario crecería indefinidamente a medida que consume contenido a lo largo de los meses. Al usar referencias cruzadas, mantenemos las colecciones de usuarios y películas ligeras y estáticas, optimizando las escrituras masivas concurrentes en una colección independiente.
- ¿Qué se perdería o ganaría si se hubiera modelado de manera relacional?

Se perderían

- **Velocidad de Lectura (Rendimiento):** En un modelo SQL, mostrar la ficha de una película con sus géneros y director requeriría hacer múltiples operaciones de **JOIN** entre tablas separadas (**Películas** $\dots$ **Directores** $\dots$ **Generos**). En MongoDB, con una sola lectura directa al documento de la película, la aplicación ya tiene toda la información lista para renderizar en pantalla.
- **Escalabilidad Horizontal:** La colección de reproducciones acumulará millones de registros rápidamente. En entornos NoSQL, esta colección se puede particionar (*sharding*) horizontalmente de forma nativa por **usuario_id**, garantizando que el sistema no se ralentice, algo sumamente complejo y costoso de lograr en bases de datos relacionales tradicionales.

Lo que se ganaría

- **Integridad Referencial Nativa:** El motor SQL controlaría de manera estricta que no se pueda insertar una reproducción con un **usuario_id** o **pelicula_id** que no existan en el sistema (evitando datos huérfanos). En MongoDB, esta validación debe ser controlada manualmente desde la capa de la aplicación.
- **Actualizaciones Masivas de Catálogo:** Si la nacionalidad de un director cambia o se corrige, en SQL se modificaría en un solo registro de la tabla **Directores**. En nuestro modelo documental, al estar embebido, requeriría actualizar todos los documentos de las películas donde figure ese director (duplicación controlada a favor del rendimiento de lectura).

Coleccion usuarios

```
{
  "_id": 1,
  "nombre": "Sebastián Lescano",
  "email": "sebastian@streambase.com",
  "plan": {
    "tipo": "premium",
    "precio_mensual": 1499,
    "fecha_inicio": "2024-01-15T00:00:00Z"
  }
}
```

Coleccion peliculas

```
{
  "_id": 1,
  "titulo": "Inception",
  "anio": 2010,
  "duracion_min": 148,
  "director": {
    "nombre": "Christopher Nolan",
    "nacionalidad": "Británico"
  },
  "generos": ["Ciencia Ficción", "Acción", "Thriller"],
  "calificacion": 8.8
}
```

Coleccion reproducciones

```
{
  "usuario_id": 1,
  "pelicula_id": 1,
  "duracion_min": 148,
  "completada": true,
  "fecha": "2025-03-10T00:00:00Z"
}
```

PARTE 3 - Aggregation Pipeline de MongoDB

El Aggregation Pipeline de MongoDB es una forma de trabajar los datos pasando por varios pasos o etapas. La idea es que los documentos entran en una especie de “tubería” (pipeline) y en cada etapa se les aplica una transformación distinta, como filtrarlos, agruparlos, ordenarlos o calcular algún dato.

Cada etapa recibe un flujo de documentos, los procesa en memoria y le pasa el resultado a la etapa siguiente. Es importante destacar que este proceso es inmutable: el pipeline no modifica los datos reales en la base de datos, solo transforma la información sobre la marcha para entregar un resultado final.

No es lo mismo que hacer una consulta normal con `find()`, porque `find()` sirve más que nada para buscar y traer documentos tal cual están. En cambio, con `aggregate()` podemos modelar los datos y sacar información mucho más elaborada, como totales, promedios, rankings o estadísticas complejas

Por ejemplo, si tenemos una colección de películas, un documento individual podría verse así:

```
{
  "titulo": "Inception",
  "generos": ["Ciencia Ficción", "Acción", "Suspense"],
  "director": {
    "nombre": "Christopher Nolan",
    "nacionalidad": "Británica"
  },
  "duracion": 148,
  "reproducciones": 15000,
  "rating": 4.8
}
```

Con una consulta (`find`), podría traer todas las películas que pertenezcan al género de "Ciencia Ficción":

```
db.peliculas.find({ generos: "Ciencia Ficción" })
```

Pero por ejemplo si quiero saber cuántas reproducciones en total han acumulado los directores dentro de ese género específico, una búsqueda simple no alcanza. Ahí es donde entra el Aggregation Pipeline utilizando sus distintas etapas:

\$match (Filtrado): Sirve para filtrar los documentos según un criterio, equivalente al `WHERE` de SQL. Como buena práctica de optimización, **\$match** siempre debe ir al principio del pipeline. De esta forma se reduce drásticamente el volumen de datos que viaja por la tubería, permitiendo además que MongoDB aproveche los índices de la colección para que la consulta sea muchísimo más rápida.

```
{ $match: { generos: "Ciencia Ficción" } }
```

\$group (Agrupación): Sirve para agrupar los documentos (equivalente al `GROUP BY` de SQL). Permite acumular y calcular valores. Aquí agrupamos por el nombre del director (usando la ruta del objeto embebido `"$director.nombre"`) y sumamos todas sus reproducciones.

```
{
  $group: {
    "_id": "$director.nombre",
    "totalReproducciones": { "$sum": "$reproducciones" }
  }
}
```

\$sort (Ordenamiento): Sirve para ordenar los resultados (equivalente al `ORDER BY` de SQL). Usar `-1` significa ordenar de mayor a menor (descendente), ideal para ver primero a los directores más vistos.

```
{ $sort: { totalReproducciones: -1 } }
```

Al unir todas las piezas en el método `aggregate()`, el código ya queda estructurado en un array con etapas secuenciales.

Primero filtra únicamente las películas de ciencia ficción; luego toma esos documentos, los agrupa por director para calcular cuántas reproducciones acumuló cada uno en ese género y, finalmente, ordena la lista para mostrarnos los directores más exitosos de mayor a menor.

```

db.peliculas.aggregate([
  {
    $match: { generos: "Ciencia Ficción" }
  },
  {
    $group: {
      _id: "$director.nombre",
      totalReproducciones: { $sum: "$reproducciones" }
    }
  },
  {
    $sort: { totalReproducciones: -1 }
  }
])

```

Conclusiones:

El Aggregation Pipeline es una herramienta indispensable cuando no alcanza con buscar datos, sino que necesitamos analizarlos. Es una solución sumamente potente porque permite estructurar las consultas complejas por etapas legibles, optimizar el uso de memoria y obtener métricas listas para consumir directamente desde el catálogo de película

PARTE 6 — Uso de IA en el desarrollo del proyecto

Herramienta utilizada

Durante el desarrollo de este proyecto utilizamos Claude.

¿Para qué lo usamos concretamente?

1. Como tutor para comprender conceptos antes de implementar

Antes de escribir cualquier línea de código, utilizamos Claude para clarificar dudas conceptuales sobre el modelo de datos. Limitado vs. Sin Límites al dominio de una plataforma de streaming, y le pedimos que justificara cada decisión de embedding y referencin en términos de las reglas vistas en clase.

2. Como asistente para escribir y depurar código

Una vez que teníamos claras las estructuras de datos, le pedimos a Claude que nos ayudara a escribir los scripts de Python (`db.py`, `seed.py`). En todos los casos, antes de aceptar el código generado:

- Lo leímos línea por línea para entender qué hacía cada parte.
- Lo ejecutamos localmente y verificamos que la salida coincidiera con lo esperado según los datos de ejemplo que nosotros mismos diseñamos.
- En algunos casos, detectamos que el pipeline necesitaba ajustes (por ejemplo, el orden de las etapas en el Pipeline 1 para que el `$limit` fuera antes del `$lookup`) y le pedimos que lo explicara antes de aceptar el cambio.

3. Como herramienta de documentación y estructuración

Usamos Claude para organizar el contenido del informe: la estructura de la Parte 2, la selección de ejemplos JSON representativos y la redacción técnica de las justificaciones de diseño. En todos los casos revisamos que la terminología usada coincidiera con la de los apuntes de clase (por ejemplo, que se mencionara explícitamente "colección asociativa", "expansión constante" y "diseño impulsado por consultas").

¿Cómo evaluamos si el código generado era correcto?

Verificación funcional: ejecutamos `seed.py` y `queries.py` en nuestra instalación local de MongoDB y comparamos la salida con los resultados esperados que habíamos calculado manualmente de antemano. Por ejemplo, sabíamos que Inception debía aparecer primero en el Pipeline 1 con 3 reproducciones, porque contamos manualmente cuántas veces aparecía en los datos de ejemplo.

Verificación conceptual: para cada etapa de cada pipeline, nos aseguramos de poder explicar con nuestras palabras qué hace esa etapa y por qué está en ese lugar. Si no podíamos explicarlo, pedíamos una aclaración antes de seguir.

Verificación con el material de clase: comparamos las decisiones de diseño sugeridas por la IA con lo que aparece en los apuntes y en los ejercicios de clase. Cuando hubo discrepancias de terminología, priorizamos la nomenclatura del curso.

Fuentes: Apuntes de clase — Bases de Datos 2 — UTN Avellaneda (2025). MongoDB Aggregation Pipeline Documentation: <https://www.mongodb.com/docs/manual/aggregation/>